

交互模板

- C++ 代码:

```
#include <cstdio>
#include <iostream>

int main() {
    for (int l = 1, r = 1000000000, mid = (l + r) >> 1, res; l <= r; mid = (l + r) >> 1) {
        std::cout << mid << std::endl;
        std::cin >> res;
        if (res == 0) {
            return 0;
        } else if (res == -1) {
            l = mid + 1;
        } else if (res == 1) {
            r = mid - 1;
        } else {
            puts("OvO, I AK IOI"); // this statement will never be executed.
        }
    }
    return 0;
}
```

- 交互特点:

- $mid = (l + r) \gg 1$ 意味着mid是右移，相当于除以2
- res在这里是存储交互机答案
- `std::cout << mid << std::endl;` 记得用endl来清除缓存
- 或者`std::cout << std::flush`
- `std::cin >> res;` 读取输入流数据
- 一般来说输出和读取都在循环里面进行若干次
- 输出答案之后剩下的都差不多

- 改进思路:

- 一般交互题考虑二分或顺序输出

-

快读

```
#define rep(i,m,n) for(int i=m;i<=n;++i)

inline int read(){
    int s = 0, w = 1;
```

```

    char ch = getchar();
    while(ch < '0' || ch > '9'){if(ch == '-')w = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9') s = s * 10 + ch - '0',ch = getchar();
    return s * w;
}
const int MAXN = 20010;
int n;
int l[MAXN], r[MAXN], f[MAXN][2];

int main(){
    n = read();
    rep(i, 1, n) l[i] = read(), r[i] = read();

}

```

快写

```

void write(int x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}

```

快速乘

```

ll qmul(ll x, ll y, ll m) { // 快速模乘: 计算 (x * y) % m, 避免大数乘法溢出
    ll z = (long double) x / m * y + 0.5L; // 将 x / m * y 转为浮点数运算, 估算商 z,
    z ≈ (x * y) / m
    u64 c = (unsigned long long) x * y - (unsigned long long) z * m; // 计算 c = (x
    * y) - z * m, 得到模 m 后的结果
    return c < m ? (ll) c : (ll) (c + m); // 如果 c < m, 直接返回 c; 否则返回 c + m,
    保证结果在 [0, m) 范围内
}

```

快速幂

```

// 快速幂: 计算 (a^n) % m, 高效处理大指数
ll qpow(ll a, ll n, ll m) {
    ll res = 1; // 初始化结果为 1
    while (n) { // 当 n > 0 时循环
        if (n & 1) res = qmul(res, a, m); // 如果 n 的最低位为 1, res = res * a %
        m
        a = qmul(a, a, m); // a = a * a % m
        n >>= 1;
    }
    return res;
}

```

```

        n >>= 1; // n 右移一位
    }
    return res; // 返回 (a^n) % m
}

```

- 记住用法:
 - 题目要求 $(a^n) \% m$, 直接调用 `qpow(a, n, m)`。
 - 题目要求大数乘法 $(x * y) \% m$, 用 `qmul(x, y, m)`。

qpow简洁版

```

using ll = long long;

ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

```

费马小定理求乘法逆元

- mod必须是质数

```

using ll = long long;

// 依赖之前的 qpow 函数
ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

// 求 a 在 mod 下的逆元
ll inv(ll a, ll mod) {
    return qpow(a, mod - 2, mod);
}

```

除法取模

- $(a * \text{qpow}(b, p-2, p)) \% p$

```
// 假设要求 (A / B) % mod
// 这里的 mod 必须是质数

ll inv_B = qpow(B, mod - 2, mod); // 算出 B 的逆元
ll ans = (A * inv_B) % mod;      // 变成乘法运算
```

扩展欧几里得求逆元

- mod 不是质数, 但满足 $\text{gcd}(a, \text{mod}) = 1$ 。
- 原理: 求解 $ax + by = 1$, 则 x 即为逆元。

```
using ll = long long;

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

ll inv_exgcd(ll a, ll mod) {
    ll x, y;
    ll d = exgcd(a, mod, x, y);
    if (d != 1) return -1; // 不存在逆元
    return (x % mod + mod) % mod; // 调整为正数
}
```

bool isPrime

```
bool is_prime(int x) {
    // 1. 处理小于2的特殊情况 (0和1不是素数)
    if (x < 2) return false;

    // 2. 核心循环: 只枚举到 sqrt(x)
    // 写法注意: 用 i <= x / i 哪怕 x 接近 INT_MAX 也不会溢出
    for (int i = 2; i <= x / i; i++) {
        if (x % i == 0) return false; // 只要发现一个因子, 就立即判定为合数
    }
}
```

```
// 3. 挺过循环没死，就是素数
return true;
}
```

素数埃氏筛 $O(n \log \log n)$

```
const int N = 1e6 + 10; // 根据题目范围调整
bool is_composite[N]; // 标记数组: false表示是素数, true表示是合数
// 也可以用 vector<int> primes 存储具体的素数

void get_primes(int n) {
    // 初始化: 0和1不是素数, 手动标记 (虽然算法里一般从2开始, 但为了严谨)
    is_composite[0] = is_composite[1] = true;

    // 核心循环
    for (int i = 2; i * i <= n; i++) { // 优化1: 只需要枚举到 sqrt(n)
        if (!is_composite[i]) {
            // 如果 i 没有被标记过, 说明 i 是素数
            // 接下来把 i 的所有倍数都标记为合数
            for (int j = i * i; j <= n; j += i) { // 优化2: 从 i*i 开始
                is_composite[j] = true;
            }
        }
    }
}

int main() {
    int n = 100;
    get_primes(n);

    for(int i = 2; i <= n; i++) {
        if (!is_composite[i]) cout << i << " ";
    }
    return 0;
}
```

欧拉筛

```
const int N = 1e7 + 10; // 10^7 级别
int primes[N], cnt; // primes存质数, cnt存质数个数
bool st[N]; // st[i]为true表示i是合数(被筛掉了), false表示是质数

void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        // 如果没有被标记过, 说明 i 是质数, 加入 primes 列表
        if (!st[i]) primes[cnt++] = i;
    }
}
```

```

// 核心循环：用当前的质数 primes[j] 去筛合数
for (int j = 0; primes[j] <= n / i; j++) { // primes[j] * i <= n
    // 1. 标记 primes[j] * i 为合数
    st[primes[j] * i] = true;

    // 2. 【最关键的一行】
    // 如果 i 能被 primes[j] 整除，说明 primes[j] 已经是 i 的最小质因子了
    // 那么 primes[j] 也一定是 primes[j] * i 的最小质因子
    // 我们必须立刻停止，避免重复筛
    if (i % primes[j] == 0) break;
}
}
}

int main() {
    int n = 100;
    get_primes(n);

    for(int i = 0; i < cnt; i++) {
        cout << primes[i] << " ";
    }
    return 0;
}

```

高精度

输入输出

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm> // max needs this usually
using namespace std;

// 读入处理: string -> vector<int> (倒序)
vector<int> s2v(string s) {
    vector<int> A;
    for (int i = s.size() - 1; i >= 0; i--) A.push_back(s[i] - '0');
    return A;
}

// 输出处理: 倒序打印
void print(vector<int> &A) {
    for (int i = A.size() - 1; i >= 0; i--) cout << A[i];
    cout << endl;
}

```

比大小** (减法前判断大小) **

```
bool cmp(vector<int> &A, vector<int> &B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; i--) {
        if (A[i] != B[i]) return A[i] > B[i];
    }
    return true; // A == B
}
```

加法

```
vector<int> add(vector<int> &A, vector<int> &B) {
    vector<int> C;
    int t = 0; // 进位
    for (int i = 0; i < A.size() || i < B.size(); i++) {
        if (i < A.size()) t += A[i];
        if (i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    if (t) C.push_back(t);
    return C;
}
```

减法 必须判断大小

```
vector<int> sub(vector<int> &A, vector<int> &B) {
    vector<int> C;
    int t = 0; // 借位
    for (int i = 0; i < A.size(); i++) {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10); // 借位处理技巧
        if (t < 0) t = 1; else t = 0;
    }
    // 去除前导0 (即vector末尾的0), 主要为了防止结果像 007 这种情况
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

乘法

```

vector<int> mul(vector<int> &A, vector<int> &B) {
    // 结果长度最多为 A+B
    vector<int> C(A.size() + B.size(), 0);

    for (int i = 0; i < A.size(); i++) {
        for (int j = 0; j < B.size(); j++) {
            C[i + j] += A[i] * B[j]; // 先累加, 最后统一处理进位
        }
    }

    int t = 0;
    for (int i = 0; i < C.size(); i++) {
        C[i] += t;
        t = C[i] / 10;
        C[i] %= 10;
    }

    // 去除前导0
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

```

除法

```

#include <algorithm> // 需要用到 reverse

// A: 被除数 (vector), b: 除数 (int), r: 余数 (引用传回)
vector<int> div(vector<int> &A, int b, int &r) {
    vector<int> C;
    r = 0;
    // 从高位 (vector末尾) 开始除
    for (int i = A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i]; // 上一位余数移位 + 当前位
        C.push_back(r / b); // 产生商的这一位
        r %= b;             // 更新余数
    }

    // C现在的顺序是 [最高位, ..., 个位], 不符合我们系统 [个位, ..., 最高位] 的规矩
    // 所以必须翻转
    reverse(C.begin(), C.end());

    // 去除前导0
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}

```


调用方法

```
int main() {
    string s1, s2;
    cin >> s1 >> s2;

    vector<int> A = s2v(s1);
    vector<int> B = s2v(s2);

    // 加法
    vector<int> C_add = add(A, B);
    print(C_add);

    // 减法 (处理负号逻辑)
    if (cmp(A, B)) {
        vector<int> C_sub = sub(A, B);
        print(C_sub);
    } else {
        cout << "-";
        vector<int> C_sub = sub(B, A); // 大减小
        print(C_sub);
    }

    // 乘法
    vector<int> C_mul = mul(A, B);
    print(C_mul);

    //=====除法=====//
    string s;
    int b;
    cin >> s >> b; // 读入大整数 s 和 普通整数 b

    vector<int> A = s2v(s); // 记得用上一条回答里的 s2v 转一下
    int r = 0; // 用于接收余数

    vector<int> C = div(A, b, r);

    // 输出商
    print(C); // 用上一条回答里的 print

    // 输出余数
    cout << r << endl;

    return 0;

    return 0;
}
```

__builtin 位运算

高效位运算，bit 库的替代品。

注意：在给 64 位整数使用时须在函数名末尾添加 `ll`。

- `__builtin_ctz(x)`：返回括号内数的二进制表示**末尾 0 的个数**。
- `__builtin_clz(x)`：返回括号内数的二进制表示**前导 0 的个数**。
- `__builtin_popcount(x)`：返回括号内数的二进制表示**1 的个数**。
- `__builtin_parity(x)`：判断括号中数的二进制表示**1 的个数的奇偶性**（偶数返回 `0`，奇数返回 `1`）。

algorithm 库

`algorithm` 库提供通用算法函数（如 `sort`、`find`、`reverse`、`max`），用于排序、搜索、遍历及容器操作，支持迭代器模式，适用于数据批量处理和竞赛编程优化。

注意：

- 库中函数基本都有 `ranges` 版本
- 基于比较的函数可传入 `cmp` 以指定比较函数。

函数	描述
<code>find(v.begin(), v.end(), val)</code>	顺序查找，其中 <code>val</code> 为需要查找的值。
<code>reverse(v.begin(), v.end())</code>	翻转序列，如数组、字符串。
<code>sort(v.begin(), v.end(), cmp)</code>	内省排序，不保持相对顺序，其中 <code>cmp</code> 为自定义的比较函数。
<code>stable_sort(v.begin(), v.end(), cmp)</code>	归并排序， 保持相对顺序 ，用法同 <code>sort</code> 。
<code>unique(v.begin(), v.end())</code>	将序列 相邻的重复元素 移动到序列末尾，返回指向去重后序列结尾的迭代器， 原容器大小不变 。与 <code>sort</code> 结合使用可以实现序列去重。
<code>shuffle(v.begin(), v.end(), rng)</code>	随机地打乱序列（需 <code><random></code> 和随机引擎 <code>rng</code> ）。
<code>nth_element(v.begin(), v.begin() + n, v.end(), cmp)</code>	按指定范围进行分类，即找出序列中 第 n 大的元素 ，使其左边均为小于它的数，右边均为大于它的数。
<code>binary_search(v.begin(), v.end(), val)</code>	二分查找。其中 <code>val</code> 为需要查找的值。
<code>merge(v1.begin(), v1.end(), v2.begin(), v2.end(), v3.begin())</code>	将 有序序列 <code>v1</code> 和 <code>v2</code> 合并到序列 <code>v3</code> 。
<code>inplace_merge(v.begin() + 1, v.begin() + m, v.begin() + r)</code>	将两个（ 已按小于运算符排序的 ） <code>[1, m)</code> 、 <code>[m, r)</code> 范围 原地合并 为一个有序序列。

函数	描述
<code>lower_bound(v.begin(), v.end(), val)</code>	在 有序序列 v 中进行二分查找，返回指向 第一个大于等于 val 的元素 的迭代器。如果不存在，则返回尾迭代器。
<code>upper_bound(v.begin(), v.end(), val)</code>	在 有序序列 v 中进行二分查找，返回指向 第一个大于 val 的元素 的迭代器。如果不存在，则返回尾迭代器。
<code>is_permutation(v1.begin(), v1.end(), v2.begin())</code>	判断序列 v1 是否为 v2.begin() 指向的序列的 排列 ，即排序后集合元素两两相同。
<code>next_permutation(v.begin(), v.end())</code>	将当前排列更改为 全排列中的下一个排列 。如果当前已经是最后一个（ 从大到小 ），返回 false 并改为第一个（ 从小到大 ）；否则返回 true 。
<code>prev_permutation(v.begin(), v.end())</code>	将当前排列更改为 全排列中的上一个排列 ，用法同 next_permutation 。
<code>includes(v1.begin(), v1.end(), v2.begin(), v2.end())</code>	判断序列 v1 是否 包含 序列 v2 （不必连续）。
<code>max(a, b)</code> 或 <code>max({a, b, ...})</code>	返回两个数或初始化列表中元素的最大值。
<code>min(a, b)</code> 或 <code>min({a, b, ...})</code>	返回两个数或初始化列表中元素的最小值。
<code>max_element(v.begin(), v.end())</code>	返回序列中 最大元素 的迭代器。
<code>min_element(v.begin(), v.end())</code>	返回序列中 最小元素 的迭代器。
<code>clamp(v, l, r)</code>	如果 v 属于 [l, r] ，则返回 v ，否则返回 最临近的边界 。
<code>is_partitioned(v.begin(), v.end(), pred)</code>	判断范围是否已按给定的 谓词 pred 划分，即所有满足 pred 的元素会在所有不满足的元素之前。
<code>partition(v.begin(), v.end(), pred)</code>	划分序列，使得 pred 返回 true 的元素位于 false 的元素之前。 不保持相对顺序 。
<code>stable_partition(v.begin(), v.end(), pred)</code>	稳定划分 ，用法同 partition 。
<code>partition_point(v.begin(), v.end(), pred)</code>	检查序列分区范围，并找到 第一个不满足 p 的元素 ，如果所有满足则返回尾迭代器。
<code>transform(v.begin(), v.end(), dest, op)</code>	应用函数 op 到输入范围，并将结果存储到 dest 指向的序列。
<code>replace(v.begin(), v.end(), old_value, new_value)</code>	以 new_value 替换 序列中所有值为 old_value 的元素。

string 库

`string` 库提供字符串处理功能（如 `find`、`substr`、`stoi`），用于文本操作、转换及解析，支持动态内存管理和高效字符串算法，适用于输入处理、数据格式化及竞赛编程中的文本需求。

- `to_string(value)`：将值 `value` 转化为字符串。
- `stoi(str)`：将字符串 `str` 转化为**有符号整数**。
- `stof(str)`：将字符串 `str` 转化为**浮点数**。

cctype 库

`cctype` 库提供字符处理函数（如 `isdigit`、`isalpha`、`toupper`），用于字符分类、转换及验证，支持基于 ASCII 的高效文本操作，适用于输入解析或格式化处理。

- `isalnum(ch)`：检查字符 `ch` 是否为**字母或数字**。
- `isalpha(ch)`：检查字符 `ch` 是否为**字母**。
- `islower(ch)`：检查字符 `ch` 是否为**小写字符**。
- `isupper(ch)`：检查字符 `ch` 是否为**大写字符**。
- `isdigit(ch)`：检查字符 `ch` 是否为**数字**。
- `ispunct(ch)`：检查字符是否为**标点符**（默认： `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~``）。

numeric 库

`numeric` 库提供数值计算函数（如 `accumulate`、`inner_product`、`partial_sum` 和 `adjacent_difference`），用于累加、内积、前缀和及差分等算法操作。

- `gcd(m, n)`：计算整数 `m` 和 `n` 的**最大公约数**。
- `lcm(m, n)`：计算整数 `m` 和 `n` 的**最小公倍数**。
- `midpoint(a, b)`：计算整数、浮点或指针 `a` 和 `b` 的**中点**。
- `lerp(a, b, t)`：如果 `t` 在 `[0, 1]` 内，则计算 `a` 和 `b` 的**线性插值**，否则为**线性外推**：`a + t * (b - a)`。
- `accumulate(v.begin(), v.end(), value)`：计算给定值 `init` 与序列中各元素的**和**。
- `iota(v.begin(), v.end(), value)`：用**按顺序递增的值**填充序列，从 `value` 开始（`++value`）。
- `partial_sum(v.begin(), v.end(), dest)`：计算序列**前缀和**，赋值给 `dest` 指向的序列。
- `adjacent_difference(v.begin(), v.end(), dest)`：计算序列**差分**，用法同前缀和。

cmath 库

`cmath` 库提供数学计算函数（如 `sqrt`、`pow`、`abs`、`sin`），涵盖数值运算、几何计算及科学问题，支持浮点和整数类型的数学操作与常用算法需求。

注意：

- 函数一般接收**浮点型输入并输出浮点型**，注意精度损失。
- 如需更高精度，可在函数名后添加 `l`（如 `sqrtl`）以指定 `long double`。

函数	描述
<code>abs(x)</code>	<code>x</code> 的 绝对值 。
<code>exp(x)</code>	<code>e</code> 的 <code>x</code> 次幂。
<code>pow(x, y)</code>	<code>x</code> 的 <code>y</code> 次幂。

函数	描述
<code>log(x)</code>	以 e 为底 x 的对数。
<code>log10(x)</code>	以 10 为底 x 的对数。
<code>log2(x)</code>	以 2 为底 x 的对数。
<code>sqrt(x)</code>	x 的平方根。
<code>cbrt(x)</code>	x 的立方根。
<code>hypot(x, y)</code>	计算 x 和 y 平方和的平方根。
<code>ceil(x)</code>	向上取整 （不小于 x 的整数）。
<code>floor(x)</code>	向下取整 （不大于 x 的整数）。
<code>round(x)</code>	四舍五入取整 （不管正负）。
<code>trunc(x)</code>	向零取整 （舍去小数部分）。

bit 库

`bit` 库提供底层位操作函数（如 `countl_zero`、`bit_ceil`、`byteswap`），用于位计数、掩码生成和字节序转换，支持高效位运算及数值优化。

注意：使用 `bit` 库需要 **C++20** 以上版本。以下函数中 **x** 为**无符号整型**。

- `has_single_bit(x)`：检查 **x** 是否为**二的整数次幂**。
- `bit_ceil(x)`：**向上取二**的整数次幂（不小于 **x** 的最小二的幂）。
- `bit_floor(x)`：**向下取二**的整数次幂（不大于 **x** 的最大二的幂）。
- `bit_width(x)`：若 **x** 非零，计算 **x** 的**二进制位数**；若 **x** 为零，返回零。
- `rotr(x, s)`：计算 **x** **右循环移位** **s** 位的结果。
- `rotr(x, s)`：计算 **x** **右循环移位** **s** 位的结果。
- `countl_zero(x)`：从**最高位**起计量**连续的 0 位**数量。
- `countl_one(x)`：从**最高位**起计量**连续的 1 位**数量。
- `countr_zero(x)`：从**最低位**起计量**连续的 0 位**数量。
- `countr_one(x)`：从**最低位**起计量**连续的 1 位**数量。
- `pop_count(x)`：返回 **x** 中**1 的个数**。

C++ 版本特性

C++98

第一个官方标准，**啥都没有**。

C++11

- **自动类型推导**：`auto` 关键字根据初始化表达式推导类型。

```
auto x = 5;           // int
auto y = 5.0;         // double
```

```
auto z = "Hello";    // const char*

vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " "; // 使用 auto 推导迭代器类型
}

# 高精度运算与素数判断代码解析
```

1. highmult 函数

功能：实现高精度乘法，计算两个大整数（以字符串形式输入）的乘积。

原理：

- 将输入字符串 **A** 和 **B** 转换为数字数组，逆序存储（低位在前）。
- 模拟手算乘法，每位相乘结果累加到结果数组 **c**。
- 处理进位，确保每位数字小于10。
- 去掉前导零，将结果数组转为字符串返回。

调用方法：

```
string result = highmult("123", "456"); // 返回 "56088"
```

使用场景：

- 处理超过 **long long** 范围的大整数乘法。
- 适合比赛中需要精确计算大数乘积的题目。

2. highplus 函数

功能：实现高精度加法，计算两个大整数（以字符串形式输入）的和。

原理：

- 将输入字符串 **A** 和 **B** 转为数字数组，逆序存储。
- 逐位相加，处理进位。
- 去掉前导零，将结果数组转为字符串返回。

调用方法：

```
string result = highplus("123", "456"); // 返回 "579"
```

使用场景：

- 处理大整数加法，适用于数据范围超大的题目。

3. jd 函数

功能：判断一个整数 n 是否为素数。

原理：

- 小于2的数不是素数。
- 2和3是素数。
- 排除2和3的倍数。
- 检查从5开始的数（步长6，检查 i 和 $i+2$ ），若 n 能被这些数整除，则非素数。

调用方法：

```
bool isPrime = jd(17); // 返回 true
```

使用场景：

- 快速判断一个数是否为素数，适用于数论相关题目。
- 适合中小范围整数（ n 不太大时效率较高）。

注意事项：

- 高精度函数输入为字符串，输出为字符串，需确保输入合法（纯数字）。
- `jd` 函数适合中小规模素数判断，大数需优化或使用高精度算法。